CAPSTONE PROJECT REPORT

Data Center Cyber-Physical Defense

AI-based anomaly detection for the IT/OT boundary, with a VAPT threat reconstruction

---

Artificial Intelligence for Cyber Security (ICS3120)

Vulnerability Assessment & Penetration Testing (ICS3102)

Manipal University Jaipur

Detail	Entry
Student name	_____
Registration number	_____
Faculty guide	_____
Submission date	_____
Source code	github.com/Aaravnauti/datacenter-security-capstone

Abstract

Data centers combine IT systems with operational technology (OT) such as uninterruptible power supplies (UPS) and cooling (HVAC). An attacker who breaches an exposed monitoring tool can pivot toward the facility network and manipulate telemetry until a thermal shutdown occurs, and signature-based defenses often miss this behavior. This project builds and evaluates an AI defense layer for that boundary and pairs it with a threat reconstruction in the style of a penetration test.

The defense layer combines an unsupervised **Isolation Forest**, trained only on benign traffic, with a class-weighted supervised **Random Forest**. Alert thresholds are tuned from precision-recall curves on a validation split of the training data, and four strategies are compared: each model alone, "either flags" and "both flag". Evaluation uses the official NSL-KDD training and test files; the test file contains 17 attack types that never appear in training.

At default thresholds the Random Forest caught only 0.62 of attacks (0.25 of those from unseen attack types), while the Isolation Forest caught 0.95 at a cost of 3,174 false alarms. Threshold tuning raised the Random Forest to 0.98 recall at 0.91 precision, and class weighting alone had almost no effect. SHAP explanations, a prediction function for raw records, an interactive dashboard, a 31-test suite, continuous integration and a Docker image complete the engineering.

The threat model maps a four-stage kill chain to MITRE ATT&CK for ICS and scores five vulnerability classes with CVSS v3.1. This report does not include results from an executed penetration-test lab, and the CVSS scores are illustrative. The benchmark contains no UPS or HVAC telemetry, so performance on facility data remains to be established.

Keywords: *anomaly detection, Isolation Forest, Random Forest, threshold tuning, explainable AI, SHAP, data center security, IT/OT convergence, MITRE ATT&CK for ICS, CVSS.*

Contents

1 Introduction

2 Background and related work

3 Threat model and VAPT reconstruction

4 System design

5 Methodology

6 Implementation and engineering

7 Results and discussion

8 Compliance mapping and enterprise architecture

9 Limitations and threats to validity

10 Conclusion and future work

References

Appendix A Reproducing the results

Appendix B Lab evidence to add

Appendix C Configuration reference

1 Introduction

1.1 Background and motivation

A modern data center is two systems in one. The IT side runs servers, hypervisors and monitoring dashboards. The OT side runs the physical plant that keeps them alive: UPS units and power distribution, and cooling controllers that hold temperatures inside a narrow band. Building Management Systems (BMS) and remote-management interfaces, such as IPMI/BMC controllers, connect the two worlds over ordinary networks.

That connection is the attack surface this project studies. If an exposed monitoring application is compromised, the attacker is one pivot away from facility-management hosts. From there, spoofed battery telemetry or altered cooling setpoints can cause an automatic thermal shutdown or power cutoff without ever touching the production servers' operating systems. Signature-based firewalls look for known malicious patterns and tend to miss abnormal-but-novel behavior, which is where statistical anomaly detection can help [2][3].

1.2 Problem statement

Design, implement and evaluate an AI defense layer that flags abnormal and malicious behavior around a data center's IT/OT boundary, explain why each alert fired, and keep the false-alarm cost realistic for a facility where a mistaken response can itself cause an outage.

1.3 Objectives

- Reconstruct a realistic cyber-physical kill chain for a data center and score its vulnerability classes (the VAPT component).
- Build an unsupervised detector for unseen behavior and a supervised classifier for known attack signatures, and compare ways of combining them.
- Evaluate honestly: on a separate test set with unseen attack types, reporting precision, recall, false alarms and missed attacks rather than accuracy alone.
- Tune alert thresholds without using the test set, and quantify the false-alarm trade-off.
- Explain individual alerts, expose the models through a prediction function, and ship the work as a tested, containerized, reproducible repository.
- Map the findings and controls to NIST SP 800-82, ISO/IEC 27001 and ISO/IEC 42001, and outline a path to production.

1.4 Scope and contributions

The project delivers a Python pipeline (data preparation, training, evaluation, explanation, inference), an interactive Streamlit dashboard, automated tests and CI, a Docker image, and supporting documentation. Its main empirical contributions are:

- A measured demonstration that **the alert threshold, not the model, dominates the false-alarm cost**: tuning moved the Random Forest from 0.62 to 0.98 recall.

- A negative result: **class weighting did not fix low recall**, because the training data is already close to balanced.
- An analysis by attack novelty showing that the two models behave very differently on attack types never seen in training (Section 7.5).

> **Scope note**
> The offensive (VAPT) component of this report is a threat reconstruction with illustrative CVSS scoring. No exploitation of a live system is part of the results, and the dataset used for the models is a network-intrusion benchmark rather than facility telemetry. Section 9 discusses what this means for the conclusions.

1.5 Report structure

Chapter 2 gives background and related work. Chapter 3 presents the threat model. Chapters 4 to 6 describe the design, method and implementation. Chapter 7 reports results. Chapter 8 maps the work to security frameworks and an enterprise architecture. Chapters 9 and 10 cover limitations, conclusions and future work.

2 Background and related work

2.1 IT/OT convergence in data centers

Facility systems were historically isolated, serial-bus installations. They are now networked for remote monitoring and efficiency, often using protocols such as BACnet and Modbus that were designed without authentication. Standards guidance for operational technology security, such as NIST SP 800-82 Rev. 3 [6], therefore stresses network segmentation, strong authentication on management interfaces, and monitoring of control traffic.

2.2 Representative weaknesses

- **Monitoring software.** CVE-2022-46169 is a critical unauthenticated command-injection flaw in the Cacti monitoring tool, rated CVSS 9.8 and fixed in Cacti 1.2.23 [10]. It illustrates how a widely deployed monitoring dashboard can give an attacker a foothold.
- **Out-of-band management.** IPMI/BMC interfaces have well-documented weaknesses, including the ability to retrieve password hashes for offline cracking under IPMI 2.0 authentication. Exposed or default-credential BMCs allow control of power and console access.
- **Unauthenticated control protocols.** BACnet and Modbus accept commands from any reachable peer unless the network restricts them, so a foothold on the right segment can change setpoints.

2.3 Machine learning for intrusion detection

The **Isolation Forest** [2] isolates points with random splits; anomalies are separated in fewer splits, so a short average path length means a higher anomaly score. It can be trained on benign data alone, which suits settings where labeled attacks are scarce. The **Random Forest** [3] averages many decision trees trained on bootstrap samples and is a strong baseline for tabular classification.

The **NSL-KDD** dataset [1] is a cleaned version of KDD Cup 99 that removes duplicate records. It is dated and does not represent modern traffic, but its official separation of training and test files, where the test file includes attack types absent from training, makes it useful for studying generalization to new attacks.

For imbalanced detection problems, precision-recall analysis is more informative than ROC curves alone [12], which is why this project selects thresholds from precision-recall curves.

2.4 Explainable AI

SHAP [4] assigns each feature an additive contribution to an individual prediction, based on Shapley values. For tree ensembles an efficient exact algorithm exists, which this project uses to show analysts why an alert fired.

2.5 Frameworks and standards

MITRE ATT&CK for ICS [5] structures the kill chain, CVSS v3.1 [9] scores severity, and NIST SP 800-82 [6], ISO/IEC 27001 [7] and ISO/IEC 42001 [8] frame the controls in Chapter 8.

3 Threat model and VAPT reconstruction

3.1 Scope and ethics

The scenario targets a representative data center in which an internet-reachable or IT-reachable monitoring tool shares network reach with facility-management hosts. Any hands-on testing belongs in an isolated lab that the tester owns and is authorized to attack. This chapter is a defensive reconstruction: it describes stages and detection opportunities and does not include exploit code.

3.2 Kill chain

Table 1. Four-stage kill chain mapped to MITRE ATT&CK, with the signal the AI layer would watch.

Stage	Technique	Adversary action	Detection signal
1. Initial access	Exploit public-facing application (T1190)	Exploit an exposed monitoring tool (Cacti-class, CVE-2022-46169)	Spike in failed logins and request rate on the monitoring segment
2. Credential harvest and lateral movement	Credential Access (TA0006), Lateral Movement (TA0008)	Read plaintext secrets from configuration files, pivot over SSH toward NUT/UPS or IPMI hosts	New internal source-to-destination pairs, connection-rate deviations
3. Subsystem manipulation	Manipulation of Control (T0831, ICS)	Spoof UPS battery telemetry or change HVAC cooling setpoints	Telemetry outside the learned benign baseline
4. Physical impact	Damage to Property (T0879, ICS)	Thermal shutdown, emergency power cutoff, hardware damage	An alert should already have fired in stages 1 to 3

3.3 Vulnerability findings and CVSS scoring

Five vulnerability classes were scored with the CVSS v3.1 equations [9]. The vectors are illustrative of each class, not measured from a lab.

Table 2. Illustrative findings with CVSS v3.1 vectors and computed base scores.

ID	Finding	Vector	Score	Severity
F-01	Unauthenticated RCE in monitoring tool	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	Critical
F-02	Plaintext credentials in configuration files	AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N	6.1	Medium
F-03	IPMI/BMC reachable from IT segment, weak credentials	AV:A/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H	9.6	Critical
F-04	Unauthenticated UPS/HVAC control writes	AV:A/AC:L/PR:N/UI:N/S:C/C:L/I:H/A:H	9.6	Critical
F-05	No IT/OT network segmentation	AV:A/AC:L/PR:L/UI:N/S:C/C:L/I:H/A:H	8.9	High

The three OT-facing findings score in the high or critical band because the vectors set **Scope: Changed** (a compromise crosses from the IT zone into the OT zone) together with a high availability impact. This is the quantitative argument for segmentation.

3.4 Remediation

Table 3. Findings mapped to remediation controls.

Finding	Primary remediation
F-01	Patch monitoring software promptly; remove internet exposure; front dashboards with VPN or SSO
F-02	Move secrets to a vault; remove plaintext credentials; restrict file permissions
F-03	Dedicated out-of-band management network; MFA; remove default credentials
F-04	Require authentication and integrity protection on control writes; reject unsigned setpoint changes; log control-plane writes
F-05	Zone and conduit model (Purdue / IEC 62443); firewall OT from IT; allow-list management stations

3.5 Status of the offensive work

Not yet evidenced

This report does not contain output from an executed penetration test. Appendix B lists the evidence (scan output, verification screenshots from an isolated lab, measured CVSS inputs) that should be collected and inserted before the VAPT component is submitted.

4 System design

4.1 Architecture

The pipeline has one data path and four consumers. Data preparation produces scaled arrays. Training fits both models, tunes their thresholds and writes everything needed for later use into an artifacts folder. Evaluation, explanation, prediction and the dashboard all read those artifacts, so the trained models and their thresholds are always used together.

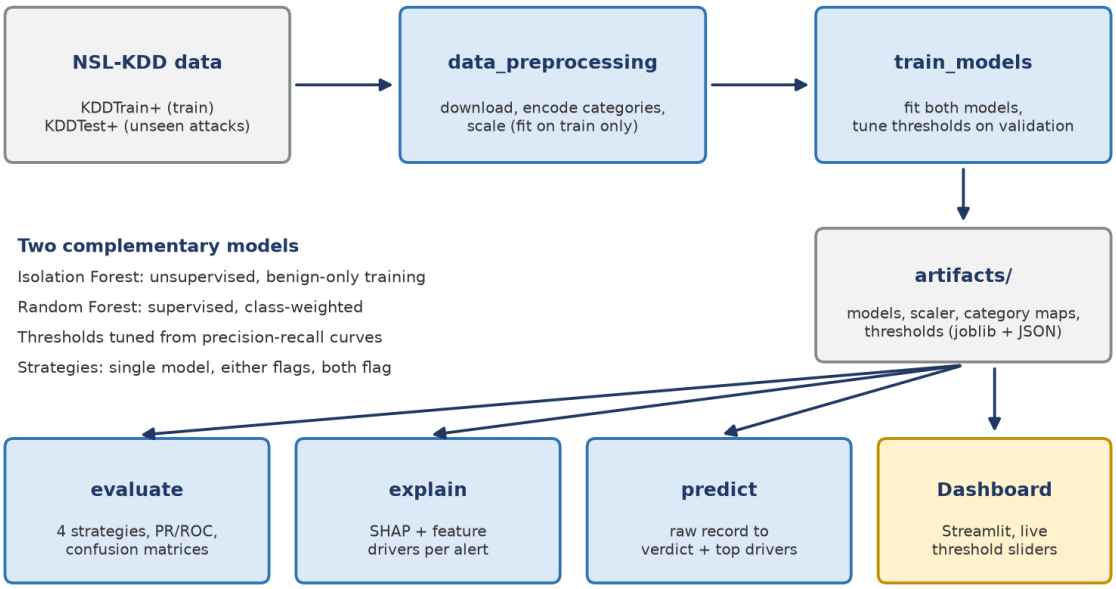


Figure 1. *System architecture: one training path producing saved artifacts, four consumers.*

4.2 Design decisions

- **Two models, not one.** The Isolation Forest needs no attack labels; the Random Forest exploits labels when they exist. Their behavior differs by attack novelty (Section 7.5).
- **Thresholds tuned on a validation split of the training data.** The test set is never used for tuning, so reported test results are not optimistic from selection. After tuning, both models are refit on all training data.
- **Scaler fit on training data only,** and Isolation Forest fit on benign rows only, to avoid leakage.
- **Persisted artifacts.** Models, scaler, category maps and thresholds are saved together so inference applies exactly the same preprocessing as training.
- **Separate official test file.** An earlier version split one file and reported a perfect score of 1.00, which was an artifact of the split. Evaluating on the separate file removed it.

4.3 Repository layout

Table 4. Source modules and their responsibilities.

Module	Responsibility
`data_preprocessing.py`	Download and cache data, encode categories (maps persisted), scale, split
`train_models.py`	Fit both models, tune thresholds, save and load artifacts
`thresholds.py`	Precision-recall threshold optimization and the four detection strategies
`evaluate.py`	Strategy metrics, PR and ROC curves, confusion matrices, novelty analysis
`feature_analysis.py`	Feature importances and drivers of flagged traffic
`explain.py`	SHAP summary plots and per-alert explanations
`predict.py`	`predict_telemetry()` for raw records, with a command-line interface
`pipeline.py`, `config.py`	One-command run; configuration and path helpers
`app.py`	Streamlit dashboard

5 Methodology

5.1 Dataset

Table 5. NSL-KDD files used. Labels are binarized: 0 is benign, 1 is any attack.

File	Role	Rows	Benign	Attacks	Attack types
KDDTrain+	Training	125,973	67,343 (53.5%)	58,630	22
KDDTest+	Test	22,544	9,711 (43.1%)	12,833	37

Each record has 41 features: 38 numeric (durations, byte counts, error rates, host-level connection statistics) and three categorical (`protocol_type`, `service`, `flag`). Seventeen of the 37 attack types in the test file never appear in training. Those 3,750 attacks make up 29.2% of the test attacks, so the test file measures generalization to new attack types.

5.2 Preprocessing

- Categorical features are mapped to integer codes using sorted category names; categories seen only in the test file are included so encoding never fails. The maps are saved for inference, and an unseen category at inference time is coded as -1 and reported.
- Numeric features are standardized with a scaler fit on training data only.
- A deterministic fixed-seed configuration makes runs repeatable.

5.3 Models

Table 6. Model configuration.

Model	Learning	Key settings
Isolation Forest	Unsupervised; fit on benign training rows only	100 trees; contamination set to the attack share of the training labels (clipped to 0.01-0.5); seed 42
Random Forest	Supervised; fit on all training rows	200 trees; `class_weight = balanced_subsample`; seed 42

For the Isolation Forest the alert score is the negative of scikit-learn's `score_samples`, so a higher score means more anomalous. For the Random Forest the score is the predicted probability of the malicious class.

5.4 Threshold tuning

A threshold turns a score into an alert. Left at defaults (the Isolation Forest's contamination cutoff and a probability of 0.50), thresholds ignore the operating cost of false alarms. The tuning procedure is:

1. Hold out 20% of the training data (stratified), and fit both models on the remaining 80%.
2. Compute each model's precision-recall curve on the held-out part.
3. Choose the threshold with the highest recall among those whose precision is at least a floor, set to 0.95. If several thresholds give the same recall, take the highest to raise the fewest alerts. If no threshold reaches the floor, fall back to the best F-beta score.

4. Refit both models on all training data and keep the chosen thresholds.

The precision floor encodes the data center's low tolerance for false alarms. The test set plays no part in any of these steps.

5.5 Detection strategies

Table 7. The four strategies compared.

Strategy	Alert when	Purpose
Isolation Forest	Anomaly score exceeds its threshold	Label-free coverage of unusual behavior
Random Forest	Probability of malicious reaches its threshold	Known-signature classification
Either flags	Either model alerts (logical OR)	Maximum coverage
Both flag	Both models alert (logical AND)	Fewest false alarms

5.6 Evaluation protocol

All results are on KDDTest+, for the malicious class: recall (share of attacks caught), precision (share of alerts that are real), F1, accuracy, and the raw counts of **false alarms** and **missed attacks**. ROC-AUC is reported as a threshold-free measure. Results are also broken down by whether an attack's type appeared in training. Accuracy alone is avoided because it hides the trade-off.

5.7 Explainability and inference

- **SHAP** (tree explainer) produces global summary plots and the top reasons for each alert in a 500-row test sample.
- **Feature analysis** ranks features by Random Forest importance and by how far flagged traffic shifts from unflagged traffic. It is a fast triage heuristic and is weaker than SHAP, and categorical codes are treated as numbers.
- **predict_telemetry()** accepts raw records (dict, Series, DataFrame or list), applies the saved encoding and scaling, and returns the verdict, both model scores against their thresholds, and the most unusual features.

6 Implementation and engineering

6.1 Stack

Python 3.11 or later with scikit-learn [11] for the models, pandas and NumPy for data, SHAP for explanations, joblib for model persistence, matplotlib for plots, PyYAML for configuration and Streamlit for the dashboard. All parameters live in a single YAML file.

6.2 Testing

A pytest suite of 31 tests runs without network access, using a synthetic data-center telemetry generator (temperature, power load, failed logins, packet rate) in place of the benchmark. It covers:

1. preprocessing: category encoding, unseen categories, scaling without leakage, determinism, download fallback;
2. thresholds: precision-floor behavior, the recall/precision trade-off, F-beta fallback, the OR and AND strategies, and metric counts;
3. training and evaluation: artifact saving and loading, class-weight configuration, auto and manual thresholds, metric outputs and files, attack-novelty analysis;
4. inference: separation of normal and attack records, input formats, strategies and error handling;
5. explanations: SHAP output files.

6.3 Continuous integration and containerization

A GitHub Actions workflow runs on every push. It lints with ruff, runs the tests on Python 3.11 and 3.12, executes an end-to-end smoke run on synthetic data, and builds and runs the Docker image. The workflow completed successfully on the repository's latest push, including the container build. The image runs as an unprivileged user and defaults to running the full pipeline.

6.4 Dashboard

A Streamlit dashboard loads the saved models (or trains them once) and presents four tabs: Detection, with live threshold sliders, a strategy switch, a precision-recall trade-off chart and a comparison table against default thresholds; Explainability, with the SHAP plots and a filterable table of per-alert reasons; Kill chain and findings; and Enterprise roadmap.

7 Results and discussion

All results use KDDTest+ (22,544 connections: 12,833 attacks and 9,711 benign). Tuned thresholds were 0.532 for the Isolation Forest (alert above) and 0.015 for the Random Forest (alert at or above). ROC-AUC was 0.940 for the Isolation Forest and 0.967 for the Random Forest.

7.1 Default thresholds

At default thresholds the models fail in opposite ways.

Table 8. Strategies at default thresholds (Isolation Forest contamination cutoff, Random Forest 0.50).

Strategy	Recall	Precision	F1	Accuracy	False alarms	Missed attacks
Isolation Forest	0.947	0.793	0.863	0.829	3,174	681
Random Forest	0.616	0.966	0.752	0.769	278	4,931
Either flags	0.947	0.793	0.863	0.829	3,174	681
Both flag	0.616	0.966	0.752	0.769	278	4,931

The Isolation Forest catches 0.95 of attacks but raises 3,174 false alarms. The Random Forest raises only 278 but misses 4,931 attacks. "Either flags" and "both flag" collapse onto one model each, because one model's alerts contain the other's at these thresholds.

7.2 Class weighting

Class weighting was the first remedy for the Random Forest's low recall. It made little difference, because the training data is already 53.5% benign and 46.5% attack and so has little imbalance to correct.

Table 9. Random Forest recall by class-weight setting.

Class weight	Thresholds	Recall	Precision	False alarms	Missed attacks
None	Default (0.50)	0.630	0.967	276	4,747
balanced	Default (0.50)	0.631	0.968	272	4,732
balanced_subsample	Default (0.50)	0.616	0.966	278	4,931
None	Tuned	0.956	0.920	1,060	559
balanced_subsample	Tuned	0.979	0.909	1,262	271

At the default threshold weighting does not help, and `balanced_subsample` is slightly worse. With tuned thresholds it adds 2.3 points of recall for 202 more false alarms, a small and real trade-off. Weighting is enabled in the final configuration for that reason.

7.3 Tuned thresholds

Table 10. Strategies at tuned thresholds (chosen on a validation split of the training data).

Strategy	Recall	Precision	F1	Accuracy	False alarms	Missed attacks
Isolation Forest	0.621	0.976	0.759	0.776	195	4,864
Random Forest	0.979	0.909	0.943	0.932	1,262	271
Either flags	0.979	0.908	0.942	0.931	1,279	271
Both flag	0.621	0.978	0.760	0.776	178	4,864

Threshold tuning raised the Random Forest's recall from 0.62 to 0.98 at 0.91 precision; false alarms rose from 278 to 1,262 and missed attacks fell from 4,931 to 271. The tuned Isolation Forest moved the other way, to high precision (0.98) and lower recall (0.62). With these thresholds "either flags" is almost identical to the Random Forest alone and "both flag" to the Isolation Forest alone.

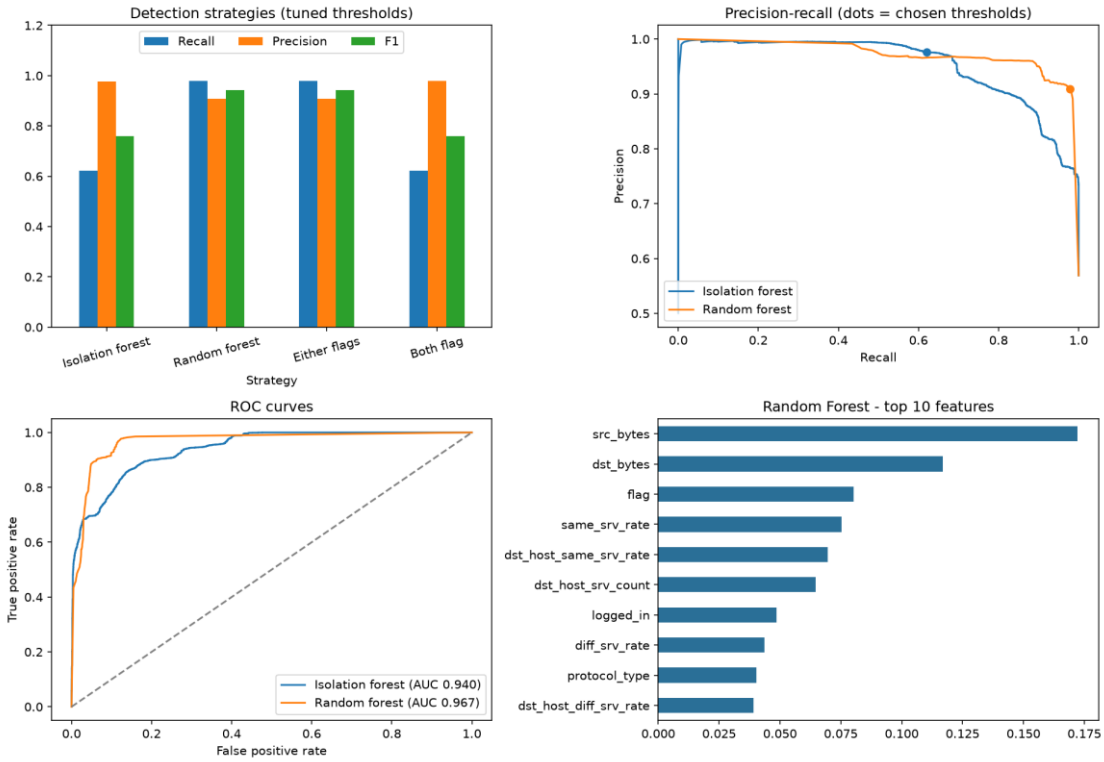


Figure 2. *Strategy comparison, precision-recall curves with chosen operating points, ROC curves and Random Forest feature importances.*

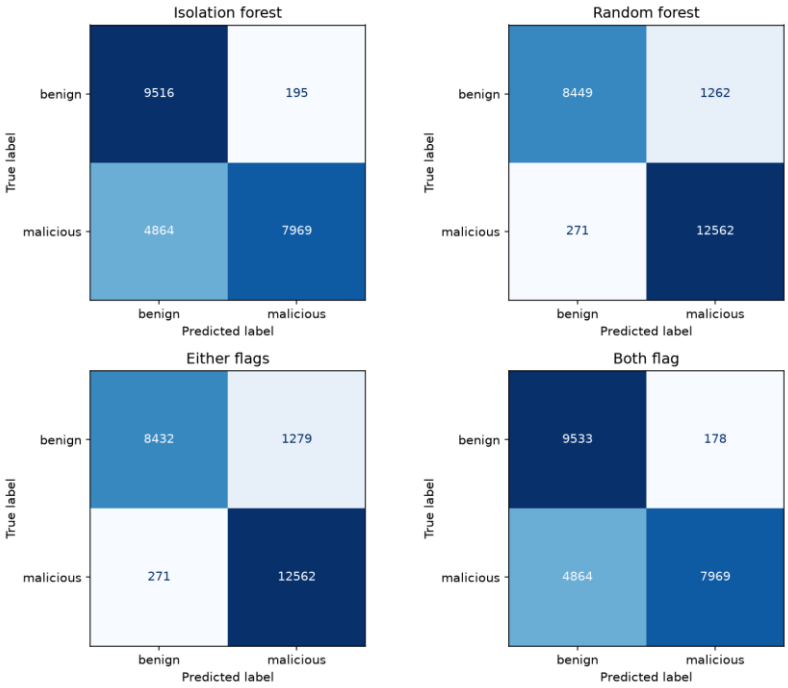


Figure 3. *Confusion matrices for the four strategies at tuned thresholds.*

7.4 Sensitivity to the operating point

The operating point is a choice, not a fixed property of the models. The table below shows the "either flags" strategy under several threshold settings, including the fixed thresholds suggested during design (Isolation Forest 0.45, Random Forest 0.40 or 0.35) and different precision floors.

Table 11. "Either flags" under different threshold settings (class-weighted Random Forest).

Setting	Recall	Precision	False alarms	Missed attacks
Default thresholds	0.947	0.793	3,174	681
Manual: IF 0.45, RF 0.40	0.803	0.920	899	2,529
Manual: IF 0.45, RF 0.35	0.805	0.919	908	2,500
Auto, precision floor 0.90	0.979	0.907	1,288	270
Auto, precision floor 0.95 (final)	0.979	0.908	1,279	271
Auto, precision floor 0.99	0.863	0.960	467	1,761

Raising the precision floor to 0.99 cuts false alarms from 1,279 to 467 at a cost of 11 points of recall. A facility can choose the point that matches its tolerance for each kind of error. Fixed thresholds of 0.35 to 0.40 for the Random Forest improved recall only modestly (to 0.67 to 0.70 for the model alone), because the useful threshold on this data is much lower, near 0.015. The sensitivity rows were computed from the same trained models with different thresholds; they were examined after the main results, and the final configuration's floor of 0.95 was fixed in advance.

7.5 Unseen attack types

Seventeen of the 37 test attack types never appear in training (3,750 attacks, 29.2% of the test attacks). Splitting recall by whether the attack type was seen exposes a difference the aggregate numbers hide.

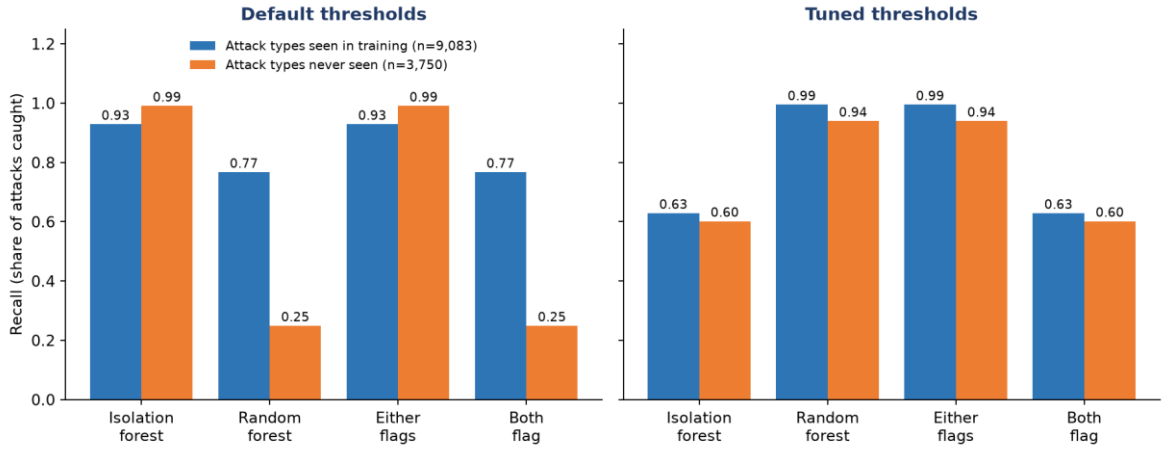


Figure 4. *Recall on attacks of types seen versus never seen in training, at default and tuned thresholds.*

Table 12. Recall by attack novelty.

Strategy	Default: seen	Default: unseen	Tuned: seen	Tuned: unseen
Isolation Forest	0.929	0.991	0.629	0.602
Random Forest	0.767	0.249	0.995	0.940
Either flags	0.929	0.991	0.995	0.940
Both flag	0.767	0.249	0.629	0.602

- At default thresholds the Random Forest catches only 0.25 of attacks of unseen types, while the Isolation Forest catches 0.99. The unsupervised detector is the better novelty detector, as its design intends.
- Tuning the Random Forest raises its recall on unseen types to 0.94, with 0.99 on seen types.
- Tuning hurts the Isolation Forest on unseen types, which fall to 0.60. Its strict threshold was chosen on validation data that contains only attack types seen in training, so it is calibrated for familiar attacks rather than novel ones.

The second point is encouraging but should be read with care. The unseen types in KDDTest+ are variants and relatives of attack families in training; a genuinely new family could behave differently. The third point suggests that, where novelty detection matters, the Isolation Forest should be given a looser threshold than a precision floor chosen on familiar attacks would produce.

7.6 Explanations and feature analysis

Random Forest importance is highest for `src_bytes` (0.172) and `dst_bytes` (0.117), followed by `flag`, `same_srv_rate` and `dst_host_same_srv_rate`. Among flagged traffic, the features that shift most from unflagged traffic, weighted by importance, were:

Table 13. Top drivers of flagged traffic (importance multiplied by shift in standard deviations).

Feature	Importance	Shift (sigma)	Driver score
flag	0.080	-1.29	0.103
dst_host_same_srv_rate	0.070	-1.18	0.082
dst_host_srv_count	0.065	-1.26	0.082
same_srv_rate	0.075	-0.95	0.071
logged_in	0.049	-1.27	0.062

SHAP summary plots show which features push each model toward an alert. For the Isolation Forest, high connection-error rates and not being logged in make traffic look anomalous.

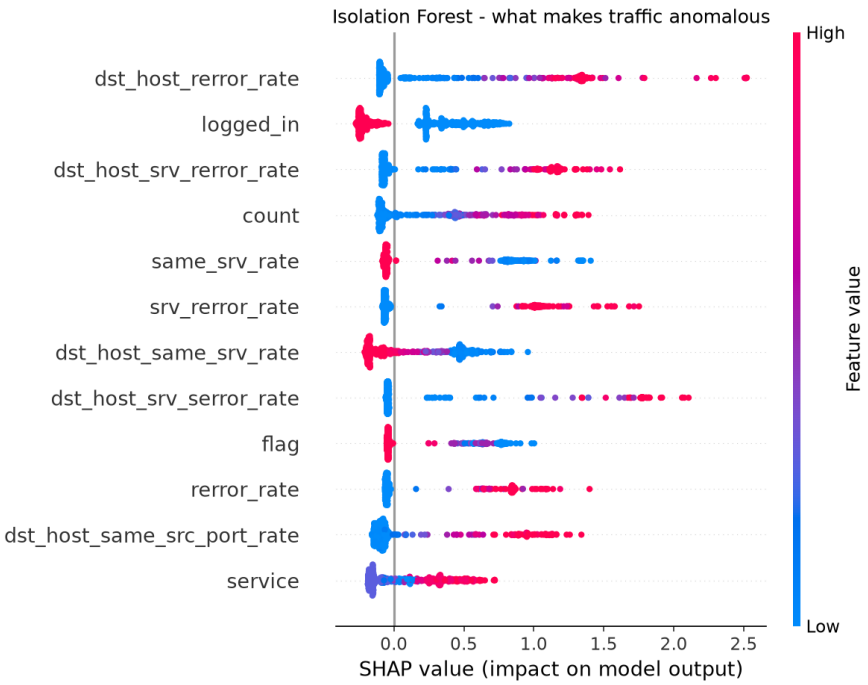


Figure 5. *SHAP summary for the Isolation Forest: what makes traffic look anomalous.*

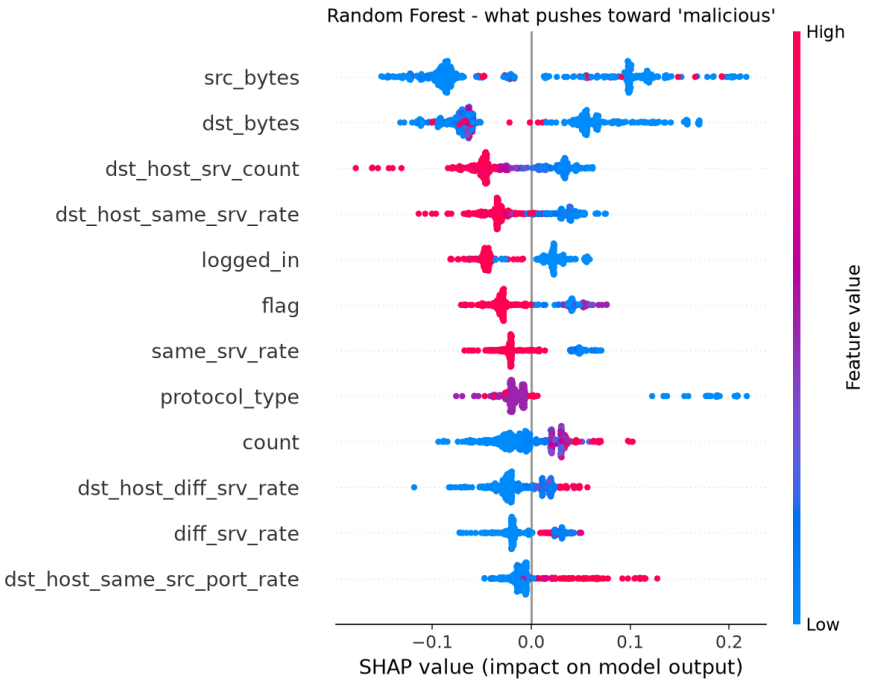


Figure 6. *SHAP summary for the Random Forest: what pushes toward the malicious class.*

7.7 Inference example

`predict_telemetry()` was run on two illustrative NSL-KDD-style records (constructed for the demonstration, not rows from the dataset).

Table 14. Inference on two illustrative records at tuned thresholds (Isolation Forest 0.532, Random Forest 0.015).

Record	IF score	RF probability	Verdict	Most unusual features
Normal web request	0.325	0.000	Benign	dst_host_srv_count, dst_host_same_srv_rate, logged_in (all within 0.6 sigma)
SYN flood	0.654	1.000	Malicious	same_srv_rate = 0.05 (-2.1 sigma), dst_host_same_srv_rate = 0.1 (-1.6 sigma), flag = S0

7.8 Discussion

- **The threshold matters more than the model.** The Random Forest's low recall was a calibration problem on novel attacks, fixed by moving its threshold, not a failure of the model family.
- **Class weighting is not a general remedy for low recall.** It helps only when imbalance is the cause, and here it was not.
- **The false-alarm cost is explicit.** The tuned Random Forest raises 1,262 false alarms in 9,711 benign connections, which is too noisy to page a person directly. A stricter precision floor, context enrichment and human review are required before any automated action.
- **The two models are complementary only in some regimes.** With tuned thresholds the Random Forest dominates; the Isolation Forest earns its place as the label-free detector and as a novelty detector at a looser threshold.

8 Compliance mapping and enterprise architecture

8.1 Findings to controls

Controls are drawn from NIST SP 800-82 Rev. 3, which applies the SP 800-53 Rev. 5 catalog to OT [6], and from ISO/IEC 27001:2022 Annex A [7]. Clause identifiers should be checked against the published standards before submission.

Table 15. Findings mapped to NIST SP 800-82 / 800-53 and ISO/IEC 27001:2022 controls.

Finding	NIST SP 800-82 / 800-53	ISO/IEC 27001:2022 Annex A
F-01 Monitoring-tool RCE	SI-2 Flaw Remediation, RA-5 Vulnerability Monitoring, CM-7 Least Functionality	8.8 Technical vulnerabilities, 8.9 Configuration management
F-02 Plaintext credentials	IA-5 Authenticator Management, SC-28 Protection at Rest	5.17 Authentication information, 8.24 Cryptography
F-03 IPMI/BMC exposure	AC-17 Remote Access, IA-2 Authentication (MFA), AC-6 Least Privilege	8.2 Privileged access, 8.5 Secure authentication
F-04 Unauthenticated control writes	SC-8 Transmission Integrity, SI-7 Integrity, AC-3 Access Enforcement	8.20 Networks security, 8.24 Cryptography
F-05 No segmentation	SC-7 Boundary Protection, AC-4 Information Flow Enforcement	8.22 Segregation of networks, 8.20 Networks security

8.2 AI defense layer to controls

Table 16. AI mechanisms mapped to controls, including ISO/IEC 42001:2023 [8].

Mechanism	NIST SP 800-82 / 800-53	ISO/IEC 27001	ISO/IEC 42001
Log and telemetry collection	AU-2, AU-6	8.15 Logging	-
Anomaly detection on telemetry	SI-4 System Monitoring	8.16 Monitoring activities	A.6 AI system life cycle (operation)
Alert response with human approval	IR-4 Incident Handling	5.24-5.26 Incident management	A.9 Use of AI systems (oversight)
Training-data provenance, poisoning guard	SI-7 Integrity	8.32 Change management	A.7 Data for AI systems
Validation before promotion	CA-2 Control Assessments	8.29 Security testing	A.6 AI system life cycle (V&V)
SHAP explanations per alert	-	-	A.8 Information for interested parties
False-positive impact assessment	RA-3 Risk Assessment	Clause 6.1	Clause 8.4 AI impact assessment

8.3 Enterprise architecture summary

The prototype scales into a production capability along four design lines, described in detail in the repository's architecture document.

- **SIEM and SOAR.** Collect Syslog, NetFlow/IPFIX, SNMPv3 and passive BACnet/Modbus telemetry through collectors into a SIEM, normalize it to a common schema, score it with a containerized inference service, and trigger SOAR playbooks. Automation may contain the IT foothold; any physical or OT action requires human approval, because an automated response to a false positive must not itself cause the outage.
- **MLOps and false-positive control.** Data versioning, a model registry with validation gates, shadow-mode deployment, drift monitoring, and layered false-positive reduction (threshold tuning, two-stage scoring, cross-checks against independent sensors, maintenance-window suppression).
- **Zero Trust segmentation.** Zones and conduits following the Purdue model, an identity-aware access layer in front of management interfaces, mutual TLS between zones, and protocol gateways or one-way gateways for legacy OT devices that cannot support mTLS.
- **Governance.** AI impact assessment, human oversight and model monitoring aligned with ISO/IEC 42001.

8.4 Roadmap

Table 17. Four-phase roadmap from prototype to production.

Phase	Timeline	Focus
1. Foundation	0-3 months	Segment IT, OT DMZ, facility and controller zones; move IPMI/BMC out of band; remediate F-01 to F-05; deploy collectors and establish a clean baseline
2. Shadow mode	3-6 months	Containerize inference; retrain on the facility's own baseline; score without alerting to calibrate thresholds against an alert budget; add SHAP to every event
3. Response	6-9 months	Go live with SOAR playbooks for enrichment and IT-side containment; human approval for OT actions; sensor cross-checks; mTLS between zones
4. Assurance	9-12 months	Drift monitoring and governed retraining; purple-team replays of this kill chain; alignment with NIST SP 800-82, ISO/IEC 27001 and 42001

9 Limitations and threats to validity

- **Benchmark, not facility data.** NSL-KDD is an old network-intrusion dataset with no UPS, HVAC or BMS telemetry. The data-center scenario motivates the work but is not measured by it. Results must be re-established on facility data, for example from public industrial-control datasets or the project's own lab.
- **Distribution shift.** Thresholds were tuned on a validation split drawn from the same distribution as training. The Random Forest threshold of 0.015 is aggressive; it transfers to KDDTest+ because the new attack types resemble training families, and may not transfer to a genuinely new family. Section 7.5 shows the same strictness hurts the Isolation Forest on unseen types.
- **Single split, single seed.** Results come from one train/test configuration and one random seed, with no confidence intervals. A leave-one-attack-family-out evaluation would test generalization more rigorously.
- **No temporal modeling.** Both models score each connection independently. Slow drift in a sensor value, which is typical of telemetry spoofing, needs a sequence model.
- **Offensive work not evidenced.** The CVSS vectors are illustrative and no lab output is included (Appendix B).
- **Operational cost not quantified.** False alarms are counted but not converted into analyst hours or outage risk.
- **Explanation quality.** SHAP was computed on a 500-row sample. The feature-driver heuristic treats categorical integer codes as numeric values, so their direction and magnitude are not meaningful.
- **Adversarial robustness.** Evasion and training-data poisoning were not tested, although the architecture document includes a poisoning guard.

10 Conclusion and future work

10.1 Conclusion

This project built an AI defense layer for the IT/OT boundary of a data center and evaluated it on NSL-KDD with a separate test file containing unseen attack types. The central finding is that the **alert threshold, not the model, decided the false-alarm cost**. At default thresholds the Random Forest caught 0.62 of attacks and the Isolation Forest 0.95. Tuning thresholds on a validation split of the training data raised the Random Forest to 0.98 recall at 0.91 precision, without touching the test set. Class weighting, the more obvious remedy, made little difference.

The analysis by attack novelty adds nuance: at default thresholds the unsupervised Isolation Forest was far better on attack types never seen in training, while the tuned supervised model matched or exceeded it, and tuning the Isolation Forest to a strict precision floor cost it that advantage. The practical lesson is to choose operating points deliberately for each model and for the cost of each kind of error.

The engineering outcome is a tested, containerized and reproducible repository with explanations, an inference function and an interactive dashboard, supported by a threat model, vulnerability scoring, compliance mapping and an enterprise roadmap.

10.2 Future work

- Run the penetration-test lab and replace the illustrative findings with measured evidence.
- Evaluate on real industrial or facility telemetry, and add sequence models such as LSTM autoencoders for temporal drift.
- Use leave-one-attack-family-out validation, multiple seeds and confidence intervals.
- Calibrate probabilities and give the Isolation Forest a dedicated novelty threshold.
- Integrate with a SIEM and a SOAR platform, and add drift monitoring and a model registry.
- Test robustness to evasion and poisoning.

References

[1] Tavallaee, M., Bagheri, E., Lu, W. and Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. IEEE Symposium on Computational Intelligence for Security and Defense Applications.

[2] Liu, F. T., Ting, K. M. and Zhou, Z.-H. (2008). Isolation Forest. IEEE International Conference on Data Mining.

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

[4] Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems 30.

[5] MITRE. ATT&CK for ICS. https://attack.mitre.org/matrices/ics/

[6] National Institute of Standards and Technology (2023). SP 800-82 Rev. 3, Guide to Operational Technology (OT) Security.

[7] International Organization for Standardization (2022). ISO/IEC 27001:2022, Information security, cybersecurity and privacy protection.

[8] International Organization for Standardization (2023). ISO/IEC 42001:2023, Artificial intelligence management system.

[9] FIRST (2019). Common Vulnerability Scoring System v3.1: Specification Document.

[10] National Vulnerability Database. CVE-2022-46169, Cacti command injection vulnerability.

[11] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[12] Saito, T. and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE, 10(3).

Appendix A Reproducing the results

From the repository root, in PowerShell or any shell:

```
python -m pip install -r requirements.txt
python -m src.pipeline          # train, tune, evaluate, explain
python -m src.predict --demo    # classify two illustrative records
python -m streamlit run app.py  # dashboard at http://localhost:8501
python -m pip install -r requirements-dev.txt
python -m pytest -q             # 31 tests
python -m ruff check .          # lint
```

Docker:

```
docker build -t datacenter-security-capstone .
docker run --rm datacenter-security-capstone
```

Outputs are written to `outputs/` (metrics, plots, SHAP files) and trained models to `artifacts/`. Set `DCSEC_DATA_SOURCE=synthetic` to run without network access.

Appendix B Lab evidence to add

The VAPT component is complete only when the following evidence, gathered in an isolated lab that the tester owns, is added to this report.

Evidence	Description	Status
Lab design	Network diagram, VM list, scope and authorization statement	To add
Reconnaissance	Port and service scan output for the monitoring host and management interfaces	To add
Vulnerability scan	Scanner output identifying the vulnerable monitoring version	To add
Exploit verification	Screenshots proving the finding in the lab (no production systems)	To add
Credential and pivot evidence	Configuration-file exposure and lateral reach toward the facility segment	To add
CVSS inputs	Measured attack vector, complexity, privileges and impact for each finding	To add
Detection linkage	Lab traffic captured during the tests, scored by `predict_telemetry()`	To add

Appendix C Configuration reference

Table 18. Key settings in config/config.yaml.

Setting	Value	Meaning
`data.source`	nsl-kdd	Data source (nsl-kdd or synthetic)
`models.isolation_forest.n_estimators`	100	Trees in the Isolation Forest
`models.isolation_forest.contamination`	from_train	Expected anomaly share, taken from the training labels
`models.random_forest.n_estimators`	200	Trees in the Random Forest
`models.random_forest.class_weight`	balanced_subsample	Penalize missed attacks
`thresholds.mode`	auto	Tune on a validation split (or use manual values)
`thresholds.validation_size`	0.2	Share of training data held out for tuning
`thresholds.min_precision`	0.95	Precision floor for threshold selection
`thresholds.manual`	IF 0.45, RF 0.40	Fixed thresholds used when mode is manual
`inference.strategy`	either_flags	Default strategy for `predict_telemetry()`